COA Module: 5-Stage RISC Pipelined Processor Simulator

Technical Report on Pipeline Hazard Resolution, Forwarding Units, CPI & Speedup Analysis

Student Name: Gautham Ram S	**Roll ID / Reg:** S70
Email: gautham.ram.s70@kalvium.community	**Program:** B.Tech Computer Science & Engineering
Live App: https://gauthamram-s70.github.io/Integrated-MicroProject-FLAT-COA-DAA/	**Repository:** https://github.com/gauthamram-s70/Integrated-MicroProject-FLAT-COA-DAA

1. Executive Summary & Curriculum Alignment

This module simulates a classic **5-Stage RISC Processor Pipeline** (Fetch, Decode, Execute, Memory, Write-Back). Aligned with **COA Modules 1–3 (Instruction Sets, Processor Organization, Pipelining and CPU Performance)**, the simulator models cycle-by-cycle instruction flow, detects and resolves Data Hazards (RAW, Load-Use), Structural Hazards, and Control Hazards, and computes key performance metrics: CPI, Speedup, and Throughput.

Curriculum Connection: *Directly applies syllabus topics 3.1–3.4 including Pipeline Throughput, Speedup formulas, Structural/Data/Control Hazard identification, and Data Forwarding multiplexer design.*

2. 5-Stage RISC Architecture Overview

Stage	Name	Hardware Operations Performed	Pipeline Registers
IF	Instruction Fetch	Fetch instruction from I-Cache via PC; increment PC ← PC + 4.	IF / ID Register
ID	Instruction Decode	Decode opcode; read source registers Rs1, Rs2 from Register File.	ID / EX Register
EX	Execute / ALU	ALU performs arithmetic/logic or computes effective memory address.	EX / MEM Register
MEM	Memory Access	Access Data Memory: Read for LOAD, write for STORE.	MEM / WB Register
WB	Write-Back	Write result back into destination register Rd in Register File.	Register File input

3. Pipeline Hazards & Resolution Mechanisms

1. RAW (Read-After-Write) Data Hazards: Occur when an instruction requires the result of an earlier instruction currently in EX or MEM.
• *Without Forwarding:* The ID stage stalls 2 cycles until the producer reaches WB.
• *With Forwarding:* Forwarding units bypass data directly from EX/MEM or MEM/WB to the ALU inputs, eliminating stalls for arithmetic sequences.
2. Load-Use Data Hazard: When a LOAD is followed immediately by an instruction using the loaded value, data is not available until MEM stage completion. Even with forwarding, a mandatory **1-cycle stall bubble** must be inserted.
3. Control Hazards (Branching): Unconditional JMP updates the Program Counter during ID/EX stage, incurring a 1-cycle branch penalty.

4. Performance Metrics & Mathematical Formulations

1. **Ideal Execution Cycles:** $C_{ideal} = k + n - 1$ (where k=5 stages, n=instructions)
2. **Actual Execution Cycles:** $C = k + n - 1 + Stalls$
3. **Cycles Per Instruction (CPI):** $CPI = C / n$ (Ideal CPI = 1.00)
4. **Sequential Execution Time:** $C_{seq} = k \times n = 5n$
5. **Speedup Factor (S):** $S = C_{seq} / C = (5 \times n) / (k + n - 1 + Stalls)$
6. **Processor Throughput (IPC):** $IPC = n / C = 1 / CPI$

5. Clock Cycle Gantt Matrix: RAW Hazard Case Study

Program: I1: LOAD R1, 50; I2: ADD R2, R1; I3: SUB R3, R2; (Forwarding Enabled - Load-Use Stall at C3)

Instruction	C1	C2	C3	C4	C5	C6	C7	C8
I1: LOAD R1, 50;	IF	ID	EX	MEM	WB	-	-	-
I2: ADD R2, R1;	-	IF	**STALL**	ID	EX	MEM	WB	-
I3: SUB R3, R2;	-	-	-	IF	ID	EX*	MEM	WB

*Note: EX stage of I3 uses EX-to-EX forwarding from I2 without stalling.

6. Performance Comparison Across Execution Modes

Execution Scenario	Instructions (n)	Total Cycles	CPI	Speedup (S)	Throughput (IPC)
Ideal Independent Pipeline	4	8	2.00	2.50x	0.500
RAW Hazards (No Forwarding)	4	12	3.00	1.67x	0.333
RAW Hazards (With Forwarding)	4	9	2.25	2.22x	0.444
Non-Pipelined Sequential	4	20	5.00	1.00x	0.200